Path Following and Tracking Errors

Contents

1. Overview . . . 2
2. The car: the kinematic bicycle model . . . 3
   - 2.1 Steering angle and wheelbase . . . 4
   - 2.2 The bicycle model's equations . . . 5
   - 2.3 Path curvature . . . 6
   - 2.4 Why the kinematic model is enough here . . . 7
3. Path following and trajectory tracking . . . 8
4. Path coordinates . . . 9
   - 4.1 The nearest path point and the distance along the path . . . 9
   - 4.2 Lateral error . . . 11
   - 4.3 Heading error . . . 12
   - 4.4 How the errors change: the model in path coordinates . . . 13
5. Cross-track error and heading error: the two errors a tracker drives to zero . . . 15
   - 5.1 Why a tracker needs both errors . . . 15
   - 5.2 Which point of the car: rear axle or front axle . . . 16
   - 5.3 Measuring a run: peak error, steady error and settling distance . . . 17
6. Where it runs: lateral and heading error in Autoware and Apollo . . . 18
7. Summary . . . 20
8. Sources . . . 21
9. Key terms . . . 22

Where this fits:

- **Builds on:** Differential-drive kinematics; Trajectory optimisation; Regulation and tracking; Driving along a line.
- **Leads to:** Pure pursuit; Rear-wheel feedback; Stanley controller; Tracking a timed trajectory (reference robot); Feedforward steering from curvature; Tracker metrics.

1. Overview

Key point: A planned path bends, so a car cannot follow it by steering towards one fixed line; at every moment it finds the nearest point of the path and measures there how far it is to the side, the lateral error, and how far its heading differs from the path's, the heading error. Steering on those two follows the whole course; a path has no clock, while a trajectory also says when to be where.

The robot can now drive to a pose: it reaches its dock at 8.45 s facing 90.0°. The robot can also drive along a line by steering on two errors, its distance from the line and its heading against the line. A planner, such as Hybrid A* for a car, gives a long path that bends, and the vehicle must stay on it.

The vehicle is now the car of RO-055, because the path trackers of this chapter were built for cars and are compared on cars (Snider §1): 2.5 m between its axles, front wheels that steer at most 26.6°, and a tightest circle of radius 5 m. A car cannot spin on the spot, so it must steer back onto a path while it drives along it. The path is a test course (Figure 1):

- 10 m straight along x from (0, 0);
- a 90° left arc of radius 10 m;
- 10 m straight up to (20, 20), 35.71 m in all.

The car starts with the middle of its rear axle at (0, 1.0), 1.0 m to the left of the course, heading 0°, and drives at 2 m/s. Figure 1 gives it RO-069's line rule with one line, the first straight. Watch the red car: it joins the line by 5 s, and at the corner, x = 10 m, it drives straight on. After 11.0 s it is at (21.94, 0.00), 5.57 m from the course and still heading 0°.

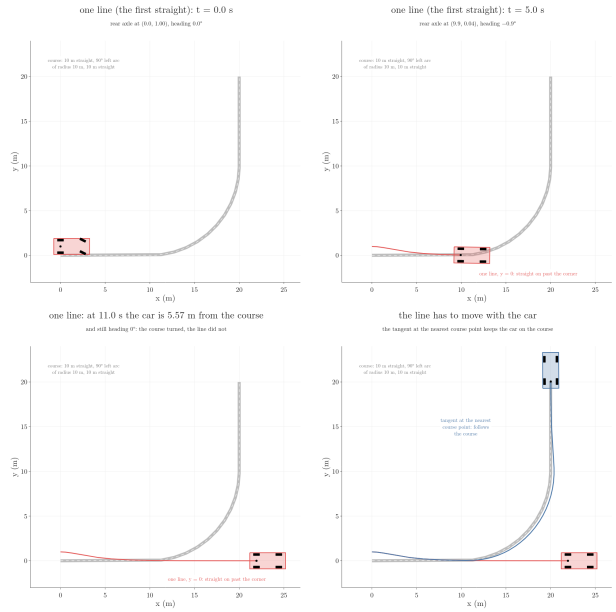


Figure 1: The test course (grey): 10 m straight, a 90° left arc of radius 10 m, 10 m straight. The car (red) starts 1.0 m left of the course at 2 m/s, steered by RO-069's line rule on one line, the first straight y = 0; one frame per 0.5 s. The car joins the line and drives straight on past the corner: at 11.0 s it is 5.57 m from the course, heading 0°. The last frame adds the same rule applied to the course's tangent at the nearest course point (blue), which follows the whole course and ends at 18.15 s

The line must move with the car. The last frame of Figure 1 uses, at every moment, the line that touches the course at its point nearest the car, and the car follows the whole course. To do that, the car must answer one question many times a second: where on the course am I, how far am I to its side, and which way do I point compared with it? This Note builds the answer:

- the car's motion, the kinematic bicycle model, and the curvature of a path (Section 2);
- following a path, which has no clock, against tracking a trajectory, which has one (Section 3);
- path coordinates: the nearest path point, the distance along the path, the lateral error and the heading error, and how they change (Section 4);
- why a tracker needs both errors, where on the car they are measured, and how to measure a run (Section 5);
- how Autoware and Apollo compute the same errors (Section 6).

2. The car: the kinematic bicycle model

> **Key point:** To steer the car back onto the course we need to know how steering moves it: a car turns about a point on its rear axle's line, the wheelbase L divided by the tangent of the steering angle δ away, so its heading turns at the speed v times the tangent of δ divided by L. At these speeds this kinematic model is enough.

2.1 Steering angle and wheelbase

Section 1's car must turn to stay on the course, and a car turns only by turning its front wheels. How sharply does a given turn of the wheels make it turn? Figure 2 draws the car from above. The two front wheels are steered together and the two rear wheels follow; for path tracking the two wheels of each axle can be drawn as one wheel in the middle of the axle, which turns the car into a bicycle (Snider §2.1, §3.1).

- The angle between the front wheel and the car's heading is the **steering angle** (G-3126) δ (delta), in rad, positive to the left; our car steers at most 26.6° either way.
- The distance from the rear axle to the front axle is the **wheelbase** (G-3127) L, 2.5 m.

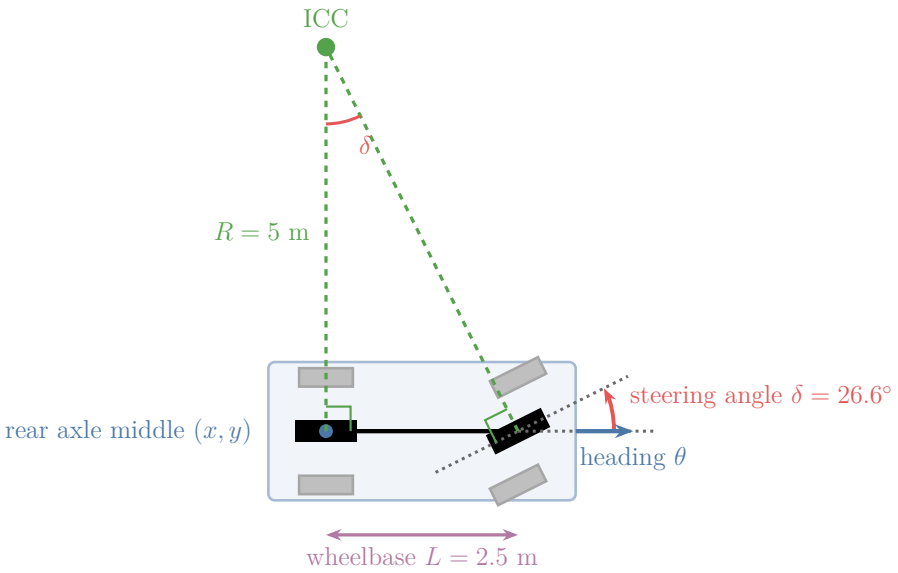


Figure 2: The kinematic bicycle model at full lock. The front and rear wheels of each axle are drawn as one wheel on the car's centre line (black); the faint wheels are the car's real ones. The rear wheel sits at the middle of the rear axle, the car's position (x, y); the front wheel is the wheelbase L = 2.5 m ahead, turned by the steering angle $\delta = 26.6°$. Each wheel rolls square to the line from the instantaneous centre of curvature (ICC, green), so the ICC lies on the rear axle's line, R = 5 m away; the angle at the ICC between the two lines is δ again

Why does the car turn about a point on the rear axle's line? Each wheel rolls without slipping, so it moves only along its own direction. As for the robot of RO-001, the whole body then turns about the one point from which each wheel's motion is square to the line joining them, the **instantaneous centre of curvature** (G-2529) (ICC). The rear wheel points along the car, so the ICC lies on the rear axle's line. The front wheel points δ off the car, so its line to the ICC leans by δ too, and the two lines meet at the ICC. In the right-angled triangle of Figure 2, the side along the car is L and the side along the rear axle is the **turning radius** (G-2293) R of the rear axle, so the tangent of δ is L divided by R (Snider eq. 1; MR §13.3.1). At full lock:

- δ is the steering angle, 26.6°, whose tangent is 0.5;
- L is the wheelbase, 2.5 m;
- R is the turning radius of the rear axle, in m.

$$\tan\delta = L/R$$

$$R = L/\tan\delta$$

$$= 2.5/0.5 = 5 \text{ m}$$

Five metres is the tightest circle of RO-055's car. A smaller steering angle gives a larger circle: the course's arc of radius 10 m needs a tangent of 2.5 divided by 10, 0.25, so 14.04°. The geometry gives the circle; the car's motion along it is the next step.

2.2 The bicycle model's equations

Knowing the circle is not yet a model: we need how fast the car's pose changes, so that we can predict it step by step as in RO-001. The car drives forward at speed v, the **linear velocity** (G-2291) of its rear axle, and it moves along its heading θ, as RO-001's robot does. A point moving at v round a circle of radius R turns at v divided by R, by RO-001's arc rule. With **dot notation** (G-2296) for rates of change:

- x and y are the position of the rear axle's middle, in m;
- θ is the car's heading, in rad;
- v is the car's speed, 2 m/s;
- δ is the steering angle and L the wheelbase, 2.5 m.

$$\dot{x} = v\cos\theta$$

$$\dot{y} = v\sin\theta$$

$$\dot{\theta} = v/R = \frac{v\tan\delta}{L}$$

These three equations are the **kinematic bicycle model** (G-3128) (Snider eqs. 8 to 10; Paden §III): RO-001's **kinematic model** (G-2297) with the turn rate set by the steering angle instead of chosen freely. Snider derives the third line from the two wheels' no-sliding rules; the ICC gives the same answer in one line. At 2 m/s:

$$\text{full lock: } 2 \times 0.5/2.5 = 0.4 \text{ rad/s}$$

the arc: $2 \times 0.25/2.5 = 0.2$ rad/s

The check at easy values: with $\delta = 0$ the tangent is 0, the heading does not turn, and the car drives straight. Figure 3 runs both cases for 8 s. Watch the green lines: they always meet at the ICC, which stays put, and each car's rear axle stays on its circle.

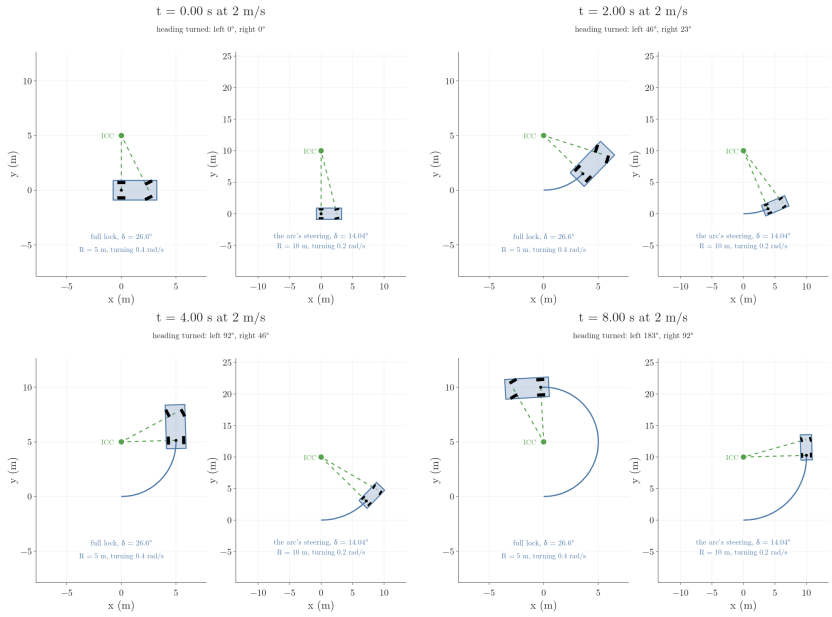


Figure 3: The kinematic bicycle model at 2 m/s, one frame per 0.25 s for 8 s. Left: full lock, $\delta = 26.6°$, the rear axle on a circle of R = 5 m, turning 0.4 rad/s. Right: $\delta = 14.04°$, the steering of the course's arc, R = 10 m, turning 0.2 rad/s, drawn at a smaller scale than the left panel to fit its larger circle. Green dashed: the lines from the ICC to the middles of the rear and front axles, square to each wheel. After 8 s the left car has turned 183°, the right car 92°

The car can turn the third equation round: to turn at a wanted rate ω at speed v, steer so that the tangent of δ is L times ω divided by v. Both cars of Figure 1 steer this way, with the turn rate from RO-069's line rule, clipped at 26.6°: 0.5 rad/s for each metre off the line and 2 rad/s for each radian of heading error. At 2 m/s these give RO-069's spring of 2 times 0.5, 1, and damper of 2, so the rule is critically damped, as in driving along a line. How much a path bends, and so how much steering it needs, is the next number.

2.3 Path curvature

The course has straights and an arc, and a car on it must steer differently on each. One number says how sharply a path bends at each point, and the same number fixes the steering the bend needs. Walk along the course's arc: over its 15.71 m, its direction turns by 90°, 1.571 rad, and it turns at the same rate all the way:

$$1.571/15.71 = 0.1 \text{ rad per m}$$

How fast a path's direction turns per metre along it is its **path curvature** (G-3129) κ (kappa), in rad per m, often written per m (Snider §3.1.1). On a circle it is 1 divided by the radius, because a full circle turns 2π rad over 2π times R:

- κ is the path curvature, 0.1 per m on the arc, 0 on the straights;
- R is the radius of the arc, 10 m.

$$\kappa = 1/R = 1/10 = 0.1 \text{ per m}$$

Putting $R = 1$ divided by κ into Section 2.1's triangle gives the steering that holds a car on a path of curvature κ: the tangent of δ is L times κ. On the arc that is 2.5 times 0.1, 0.25, the 14.04° of Figure 3's right car. The sign follows the bend: positive curvature for a left bend, negative for a right one. Curvature also gives the car's tightest path: 1 divided by 5 m, 0.2 per m, is the most this car can follow. One check remains: whether a model without tyre slip holds at our speeds.

2.4 Why the kinematic model is enough here

The bicycle model assumes that the tyres never slide sideways. Real tyres do slide a little, more the harder the car corners, so we must check that our runs corner gently. A car on a circle accelerates towards its centre, sideways, by its speed squared divided by the radius (OpenStax §4.4, centripetal acceleration). On the 10 m arc:

$$\text{at 2 m/s: } 2^2/10 = 0.40 \text{ m/s}^2$$

$$\text{at 5 m/s: } 5^2/10 = 2.50 \text{ m/s}^2$$

Polack et al. compared the kinematic bicycle model with a detailed vehicle model on curves of constant radius; their 2018 paper reports the result: the model stays valid while the sideways acceleration stays below half the road's friction (grip) coefficient times gravity. Rubber on dry concrete grips with a coefficient of about 1.0 (OpenStax §6.2, Table 6.1):

$$0.5 \times 1.0 \times 9.81 = 4.91 \text{ m/s}^2$$

Our fastest case, 5 m/s on the arc, uses about half of that, and no run in this Note goes above 2.5 m/s^2 (notebook), so the kinematic model is enough here. Snider (§2.1) states the same in words: the model "approximates the motion of a car reasonably well at low speeds and moderate steering angles". With the car's motion known, the next question is what "on the path" should mean.

3. Path following and trajectory tracking

> **Key point:** The course of Figure 1 says where to drive, not when: following it, the car only has to be close to some point of it, while tracking a trajectory it must be at a given point at a given time. After a late start, the path follower simply arrives 1.80 s late, while the tracker speeds up to 5 m/s and catches its timetable within 3.4 s.

The car of Section 1 drove the course at 2 m/s, but nothing in the course asked for 2 m/s. Think of two everyday cases. A car on a road must stay on the road; when it gets to the next town is up to the driver. A train must also stay on its track, but it must reach each signal at the scheduled moment, or it meets a red light. The car follows a path; the train tracks a trajectory.

A path is a curve in space with no clock: the course is the same whatever the speed. Keeping a vehicle close to a path, at a speed of its own choosing, is **path following** (G-3130). A **trajectory** (G-2886) adds the clock: it says where the vehicle should be at each moment. Making a vehicle be at each point of a trajectory at its time is **trajectory tracking** (G-3131), the **tracking** (G-2954) of RO-059, a reference that changes over time, applied to the whole pose. A parked car being steered into a slot, with a goal but no route, is the third case, **regulation** (G-2952), which RO-069's docking was.

Paden et al. (§V) state the difference precisely. Path following asks that the error to some point of the path, which may slide along it, shrinks to zero, while the car's progress along the path tends to a wanted speed. Trajectory tracking asks that the error to the reference point *of the present moment* shrinks to zero. The trade-off fits in two lines (UW CSE 478 L10): a timed trajectory lets us respect time limits, while an untimed path conveys the shape but cannot control when the robot reaches a point.

The difference shows when something delays the car. In Figure 4, both cars are held at the start for 1.5 s while the timetable, the orange dot, leaves at 2 m/s. The timetable is the course driven at 2 m/s from $t = 0$, so it reaches the end at 17.85 s. The gap along the course between where the timetable says the car should be and where it is, is the **along-track error** (G-3137) (UW CSE 478 L10). At 1.5 s it is 3.0 m for both cars. Then:

- the path follower (blue) drives at 2 m/s, steering with the line rule of Figure 1. How far it is from the course at each point along the course is exactly the same as without the hold; it ends at 19.65 s, 1.80 s behind the timetable, with an along-track error of 3.59 m, and that is not a fault for a path follower;
- the trajectory tracker (purple) steers the same way and also sets its speed from the along-track error: 2 m/s plus 1 m/s for every metre behind, at most 5 m/s, so it starts at 5.0 m/s. Its along-track error is within 0.1 m by 4.89 s, and it ends at 17.90 s, 0.092 m behind the timetable.

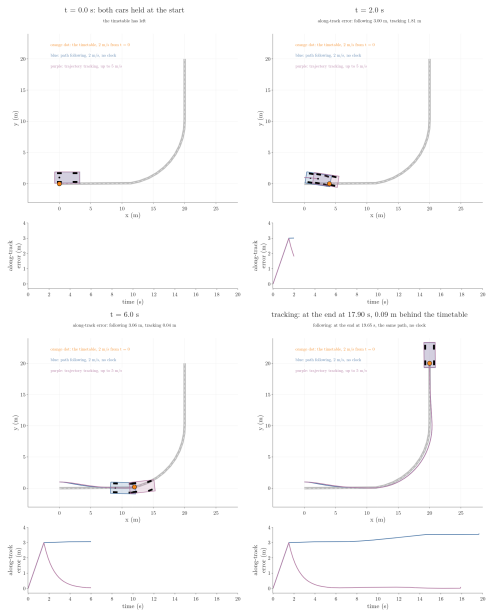


Figure 4: Path following against trajectory tracking, one frame per 0.5 s. Both cars are held at the start for 1.5 s; the orange dot is the timetable, a point moving along the course at 2 m/s from t = 0. Blue: path following at 2 m/s, no clock. Purple: trajectory tracking, speed 2 m/s plus 1 m/s per metre of along-track error, at most 5 m/s. Lower panel: each car's along-track error, the timetable's distance along the course minus the car's. The tracker's error falls from 3.0 m to within 0.1 m by 4.89 s, and it ends at 17.90 s; the follower stays 3.0 to 3.6 m behind and ends at 19.65 s

Both cars need the same two numbers to stay on the course: how far they are to its side and how their heading differs from its direction. The tracker needs one more, the along-track error. The trajectory tracker here only illustrates the difference; a law that handles all three errors together is tracking a timed trajectory, later in this chapter. Both cars first need a point of the course to measure from, which Section 4 finds.

4. Path coordinates

> **Key point:** For a car near a path, the nearest point of the path gives three numbers: s, how far along the path that point is; e, how far the car is to the side of it, positive to the left; and θ_e, the car's heading minus the path's direction there. On the course's arc, the car at (14.0, 0.0) heading 10° has $s = 13.81$ m, $e = -0.770$ m and $\theta_e = -11.80°$.

4.1 The nearest path point and the distance along the path

The car of Figure 1 needed the course's tangent at "the point nearest the car". Which point is that, and how far along the course does it lie? Take the car with its rear axle at (14.0, 0.0), heading 10°, beside the course's arc (Figure 5).

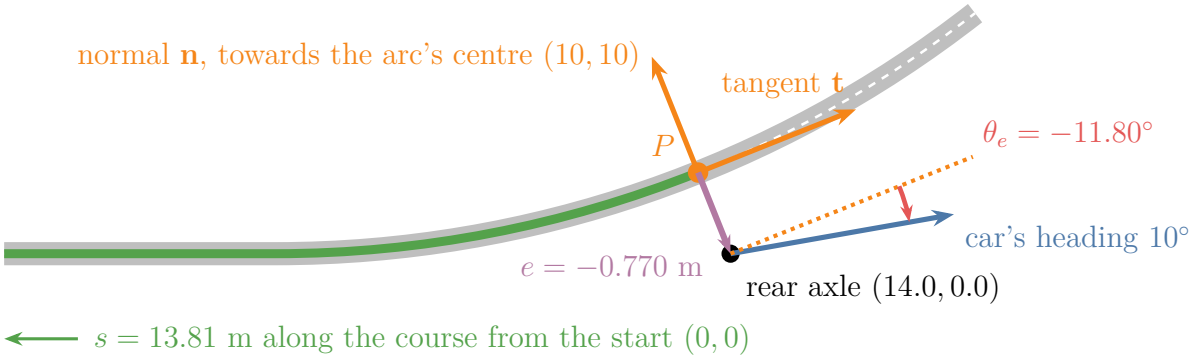


Figure 5: Path coordinates of the car with its rear axle at (14.0, 0.0), heading 10°, beside the course's arc (centre (10, 10), radius 10 m). P (orange) is the course point nearest the rear axle, (13.714, 0.715), on the line from the arc's centre through the car. Green: the distance along the course from the start (0, 0) to P, s = 13.81 m. At P the tangent t and the normal n, which points to the left of the tangent and, on this arc, at its centre. Purple: the lateral error e = −0.770 m from P to the rear axle (negative: right of the course). Red: the heading error, from the course's direction at P (dotted copy at the car, 21.80°) to the car's heading, 10°: −11.80°

The nearest point P is where a circle round the car first touches the course. On a straight, it is where a line square to the straight passes through the car: for the car at its start, (0, 1.0), it is (0, 0). On an arc, the nearest point lies on the line from the arc's centre through the car, at the arc's radius from the centre, because every other arc point is farther away. With the arc's centre at (10, 10):

$$\text{car minus centre} = (4.0, -10.0)$$

$$\text{distance} = \sqrt{4.0^2 + 10.0^2} = 10.770 \text{ m}$$

$$P = (10, 10) + 10 \times (4.0, -10.0)/10.770$$

$$= (13.714, 0.715)$$

The course point nearest a given point of the car, here its rear axle, is the **nearest path point** (G-3134) (Paden §V; Snider §3.1.1); trackers also call it the closest point or the matched point.

How far along the course is P? A path can be laid out by the distance walked along it from its start, like the kilometre stones of a road. That distance is the **arc length** (G-3133) s, in m (Snider §3.1.1). P lies after the 10 m straight and partway round the arc. Seen from the

centre, P is at $-68.20°$, and the arc began at $-90°$, so the arc has swept 21.80°, 0.3805 rad, and by the arc rule:

$$s = 10 + 10 \times 0.3805$$

$$= 13.81 \text{ m}$$

With s known, the course's direction at P is known too: the arc began heading 0° and has turned 21.80°, so it heads 21.80°. At P we put two arrows: the tangent $\mathbf{t}$, along the course's direction, and the normal $\mathbf{n}$, square to it on the left. Measuring a point's place by the distance along a path and the distance to its side, in these two directions, gives the **path coordinates** (G-3132) of the point. The tangent and normal at each path point form a small frame that slides along the path, the Frenet frame of the curve, so path coordinates are also called Frenet coordinates (Snider §3.1.1; MathWorks `referencePathFrenet`). The first number, s, says where along the course the car is; the next two say how wrong it is there.

4.2 Lateral error

The car of Figure 5 is beside P, not on it. How far, and on which side? The signed distance from the course to the car, along the normal at P, positive when the car is to the left of the course's direction, is the **lateral error** (G-3135) e, in m (Snider §3.1.1). Driving a vehicle along a path, it is also called the **cross-track error** (G-3135), the error across the track (Snider §2.2; UW CSE 478 L10). On the arc it is the radius minus the car's distance from the centre, because the left side of a left arc is the inside:

$$e = 10 - 10.770 = -0.770 \text{ m}$$

The minus sign says the car is 0.770 m to the right of the course, on the outside of the arc. At the start, the car at (0, 1.0) has P = (0, 0) and $e = 1.0$ m, to the left.

For a path that is not an arc, the same number comes from the offset of the car from P, $dx = 0.2861$ m and $dy = -0.7152$ m, and the course's heading at P, $\theta_p = 21.80°$. The lateral error is the part of the offset along the normal, the normal being the tangent's direction turned left by 90° (Apollo, Autoware; Section 6):

- dx and dy are the car's position minus P's, in m;
- θ_p is the path's heading at P, in rad.

$$e = -\sin\theta_p \, dx + \cos\theta_p \, dy$$

$$= -0.3714 \times 0.2861 + 0.9285 \times (-0.7152)$$

$$= -0.1063 - 0.6641$$

$$= -0.770 \text{ m}$$

Both ways agree. The check at easy values: on the first straight θ_p is 0, its sine 0 and its cosine 1, so e is just dy, the height above the line, 1.0 m at the start. The lateral error says how far off the course the car is; it does not say whether the car is getting back.

4.3 Heading error

A car 0.770 m outside the arc may be heading back towards the course or away from it. The car's heading minus the path's heading at P, wrapped into $-180°$ to $180°$ by **angle wrapping** (G-2404), is the **heading error** (G-3136) θ_e, in rad, positive when the car points to the left of the path (Snider §3.1.1):

- θ is the car's heading, 10°;
- θ_p is the path's heading at P, 21.80°.

$$\theta_e = \theta - \theta_p$$

$$= 10° - 21.80° = -11.80°$$

The car points 11.80° to the right of the course: further out, away from the arc's centre, so its lateral error is growing more negative. These three numbers, s, e and θ_e, are what the last frame of Figure 1 used at every step. Figure 6 replays that run with all three drawn live. Watch the purple arrow, the lateral error: it starts at 1.000 m, shrinks to within 0.1 m by s = 7.70 m, and on the arc settles near -0.39 m, while the red heading error, after its swing to $-10.7°$ at the start, stays near 0.

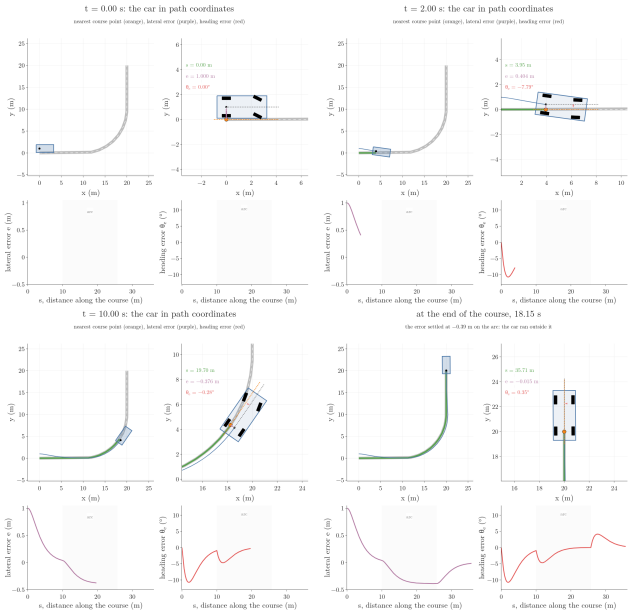


Figure 6: Path coordinates live, one frame per 0.25 s: the car at 2 m/s steered by RO-069's line rule on the course's tangent at the nearest course point. Top left: the whole course, with the distance s along it so far in green. Top right: a 10 m by 8 m window that follows the car, with the nearest course point (orange), the course's direction there (orange dotted, and a grey dotted copy at the car), the lateral error from that point to the rear axle (purple) and the heading error from the course's direction to the car's heading (red arc). Lower panels: the lateral error e and the heading error θe against s; the grey band is the arc. The last frame: the end of the course at 18.15 s; on the arc the error settled near −0.39 m, outside the arc

4.4 How the errors change: the model in path coordinates

A tracker steers to make e and θ_e shrink, so it needs to know how its steering changes them. The bicycle model of Section 2.2 says how x, y and θ change; rewritten in s, e and θ_e it says how the path coordinates change (Snider eq. 12; Paden eq. V.10). Take the car of Figure 5 at 2 m/s, steering the arc's 14.04°:

- the part of its velocity along the course, v times the cosine of θ_e, carries it round the arc's centre at a radius of 10.770 m, and the nearest point P, at the arc's 10 m radius, sweeps round at the same angular rate, so P moves along the course a little more slowly than the car;
- the part across the course, v times the sine of θ_e, changes the lateral error, as it changed the distance from a line for the room robot;
- the heading turns at the bicycle model's rate, and the course's direction at P turns at κ times the speed of P along the course; the heading error changes by the difference.

The symbols, with the values of the car of Figure 5:

- v is the car's speed, 2 m/s;
- δ is the steering angle, 14.04°, whose tangent is 0.25, and L the wheelbase, 2.5 m;
- κ is the path curvature at P, 0.1 per m;

- e is the lateral error, -0.770 m, and θ_e the heading error, $-11.80°$;
- $\dot{s}$, $\dot{e}$ and $\dot{\theta}_e$ are the rates of change of s, e and θ_e.

Our numbers first:

$$v \cos\theta_e = 2 \times 0.9789 = 1.958 \text{ m/s}$$

$$\text{about the centre: } 1.958/10.770$$

$$= 0.1818 \text{ rad/s}$$

$$\dot{s} = 10 \times 0.1818 = 1.818 \text{ m/s}$$

$$\dot{e} = 2 \times \sin(-11.80°)$$

$$= -0.409 \text{ m/s}$$

$$\dot{\theta}_e = 2 \times 0.25/2.5 - 0.1 \times 1.818$$

$$= 0.2000 - 0.1818 = 0.0182 \text{ rad/s}$$

The same steps in symbols, with the radius $R = 1$ divided by κ, so the car is at R minus e from the centre:

$$\dot{s} = R \, \frac{v \cos\theta_e}{R - e}$$

$$= \frac{v \cos\theta_e}{1 - \kappa e}$$

$$\dot{e} = v \sin\theta_e$$

$$\dot{\theta}_e = \frac{v \tan\delta}{L} - \kappa \, \dot{s}$$

Snider derives the same three lines for any path whose curvature changes along it, with κ the curvature at P. The check at easy values: a car on the course and aligned with it, $e = 0$ and

$\theta_e = 0$, has $\dot{s} = v$ and $\dot{e} = 0$, and its heading error stays 0 exactly when the tangent of δ is L times κ, the steering of Section 2.3. Our car, 0.770 m outside the arc and pointing out, drifts out at 0.409 m/s while its heading error rises towards 0 by only 0.018 rad/s: holding the arc's steering will not bring it back. The equations say what each error does; Section 5 asks what a tracker must do with them.

5. Cross-track error and heading error: the two errors a tracker drives to zero

> **Key point:** The lateral error, how far the car is to the side of the path, and the heading error, how far its heading differs from the path's, are the two numbers a path tracker drives to zero: with only the first the car weaves across the course, up to 2.09 m off, and with only the second it stays 1.00 m off. Where on the car they are measured matters: at 1.0 s the rear axle is 0.737 m off and the front axle 0.271 m.

5.1 Why a tracker needs both errors

Figure 6's car brought both errors near zero. Would one of them be enough? The model of Section 4.4 suggests not: the lateral error changes only through the heading error, at the speed times the sine of the heading error, so the heading error is what the car must manage to fix the lateral error. RO-069's line rule turned the two into a spring and a damper: the lateral-error term is the spring, which pulls the car back, and the heading-error term is the damper, which stops it overshooting. Figure 7 removes each in turn, from the same start, 1.0 m left of the course at 2 m/s:

- lateral error only (red, heading gain 0): a spring with no damper swings for ever. The car crosses the course at $s = 3.16$, 9.62, 14.56, 24.50 and 30.79 m and swings up to 2.09 m off;
- heading error only (purple, lateral gain 0): nothing pulls the car back. The car turns parallel to the course and stays 1.00 m off along the whole first straight; on the arc it drifts outwards, because there it can turn with the course only while its heading error is negative;
- both errors (blue): within 0.1 m by $s = 7.70$ m on the straight.

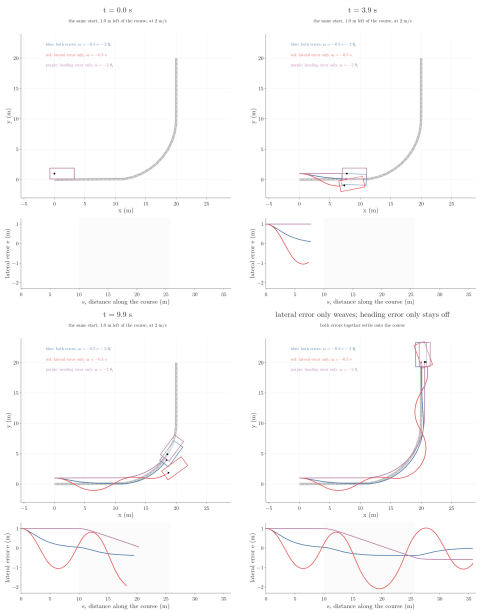


Figure 7: Why a tracker needs both errors, one frame per 0.3 s; three cars from the same start, 1.0 m left of the course at 2 m/s. Blue: the line rule on both errors, turn rate $\omega = -0.5\,\mathrm{e} - 2\,\theta\mathrm{e}$, with e the lateral error and θe the heading error. Red: lateral error only, $\omega = -0.5\,\mathrm{e}$: it weaves across the course, up to 2.09 m off. Purple: heading error only, $\omega = -2\,\theta\mathrm{e}$: it stays 1.00 m off along the first straight and drifts outwards on the arc. Lower panel: the lateral error e of each against the distance s along the course

These two numbers are what every path tracker steers on (UW CSE 478 L10: "some control laws will only minimize cross-track error, others will also minimize heading"). Each tracker in the next Notes combines them in its own way. Before that, one detail changes their values: the point of the car they are measured at.

5.2 Which point of the car: rear axle or front axle

So far every error was measured at the middle of the rear axle, the car's position. A tracker may measure at another point, and the numbers change. Figure 8 takes the car 1.0 s into Figure 6's run, still on the first straight: its rear axle is 0.737 m left of the course and it heads $-10.74°$, back towards the course. Its front axle is 2.5 m ahead along the heading, so on a straight path its lateral error differs by the wheelbase times the sine of the heading error; Apollo's lateral controller uses the same rule to estimate the error at a point ahead of the car (`lateral_error_feedback`, Section 6):

- e is the lateral error at the rear axle, 0.737 m;
- e_f is the lateral error at the front axle, in m;
- L is the wheelbase, 2.5 m;
- θ_e is the heading error, $-10.74°$.

$$e_f = e + L \sin \theta_e$$

$$= 0.737 + 2.5 \times \sin(-10.74°)$$

$$= 0.737 - 0.466 = 0.271 \text{ m}$$

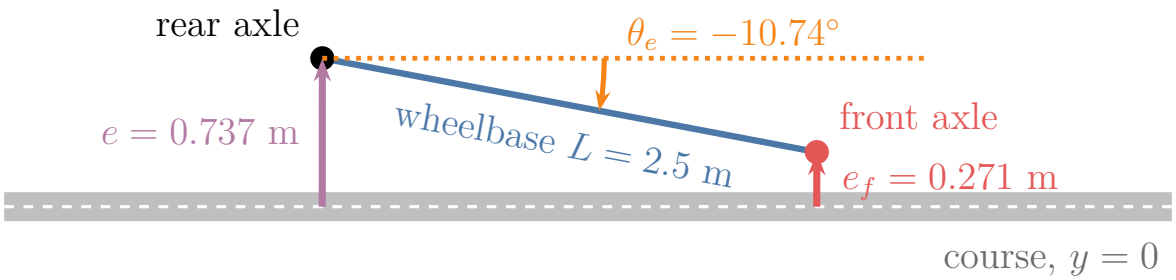


Figure 8: The same car 1.0 s into the run, on the first straight, measured at two points. The rear axle (black) is e = 0.737 m left of the course; the heading error is −10.74°, so the front axle (red), the wheelbase L = 2.5 m ahead, is only ef = 0.271 m off, because the car points back towards the course

The front axle's error already contains part of the heading error, the wheelbase times its sine. Each tracker fixes its point: the Stanley controller, met two Notes on, measures at the front axle, while pure pursuit and rear-wheel feedback measure at the rear axle (Snider §2.2, §2.3; Paden §V.A). Apollo's lateral controller measures at a third point, the car's centre of mass (Section 6). Whatever the point, it must be stated, because the same car has a different lateral error at each. With the point fixed, a whole run can be summed up in a few numbers.

5.3 Measuring a run: peak error, steady error and settling distance

To compare trackers, the next Notes need a few numbers that sum up a whole run of Figure 6. Each answers a question a driver would ask:

- How far off did it get? The largest lateral error, here 1.000 m, at the start.
- How soon was it back? The distance along the course at which the lateral error first falls within 0.1 m: $s = 7.70$ m, after 3.88 s.
- Did it stay on the bend? The error the car keeps while it drives the arc, once it has settled: near −0.39 m (at most −0.388 m), outside the arc.

Why does the car keep a steady error on the arc? To follow the arc it must turn at its speed v, 2 m/s, times the arc's curvature κ, 0.1 per m:

$$2 \times 0.1 = 0.2 \text{ rad/s}$$

RO-069's line rule turns the car only in proportion to its errors. With the heading error settled at about 0, the turn must come from the lateral error alone:

$$-0.5\,e = 0.2$$

$$e = -0.4 \text{ m}$$

The simulated -0.388 m is a little smaller, because outside the arc the nearest point moves more slowly than the car, so the course turns at only 0.1925 rad/s (Section 4.4). The steady error on the arc is the same kind of error that a proportional controller leaves under a steady load, the **steady-state error** (G-2985) of RO-061. After the arc the error is back within 0.1 m from $s = 30.92$ m. These three numbers, and the run's 18.15 s, are the yardstick the next Notes use, measured at the rear axle.

6. Where it runs: lateral and heading error in Autoware and Apollo

> **Key point:** Autoware and Apollo, two open software stacks for self-driving cars, compute the errors of Section 4 in a few lines: find the nearest point of the planned trajectory, then rotate the car's offset from it into the path's direction. Both can also pick the reference point by time instead, which turns path following into trajectory tracking.

Every tracker in a real vehicle starts with the nearest point and the two errors. In Autoware's lateral controller (`control/autoware_mpc_lateral_controller` in the `autoware_universe` repository, read for this Note):

- `calcNearestPoseInterp` in `src/mpc_utils.cpp` finds the nearest stored trajectory point, then places the nearest pose between it and its neighbour, so it is not limited to the stored points. The search (`findFirstNearestIndexWithSoftConstraints`, from `autoware_motion_utils`) first looks only at points within a distance and a heading limit of the car, and drops the limits only if no point meets them; the README of `autoware_motion_utils` draws the cases this guards against, such as a path that runs back beside itself or crosses itself. Paden et al. (§V) explain the danger: on such a path the plain nearest point can jump from one part to another.
- `calcLateralError` computes the lateral error exactly as in Section 4.2: `lat_err = -std::sin(ref_yaw) * err_x + std::cos(ref_yaw) * err_y`, with the car's offset (`err_x`, `err_y`) from the nearest pose and that pose's heading `ref_yaw`.
- The heading error in `src/mpc.cpp` is `yaw_err = normalize_radian(current yaw - nearest yaw)`, the heading error of Section 4.3, wrapped.
- The parameter `trajectory_reference_mode` chooses how the reference point is found: `"spatial"`, the default, takes the nearest pose, which is path following; `"temporal"` takes the pose the trajectory gives for the time elapsed, which is trajectory tracking (Section 3).

Apollo's lateral controller (the function `ComputeLateralErrors` in `lat_controller.cc`, in the folder `lat_based_lqr_controller` under `modules/control/controllers` of the `ApolloAuto/apollo` repository) does the same:

- it finds the reference point with `QueryNearestPointByPosition`, a search for the stored point closest to the car, which relies on the points being dense; with the setting `query_time_nearest_point_only` it takes the point for the present time instead;
- `lateral_error = cos_target_heading * dy - sin_target_heading * dx` and `heading_error = NormalizeAngle(theta - ref_heading)`, the same two formulas;
- it also computes the rates of Section 4.4: `lateral_error_rate = v * sin(heading_error)`, and the heading error rate as the car's turn rate minus the path's curvature times the reference speed;
- it estimates the lateral error at a point ahead along the car's heading with the rule of Section 5.2: `lateral_error_feedback = lateral_error + lookahead_station * sin(heading_error)`, where `lookahead_station` is that point's distance ahead;
- outside its navigation mode it measures at the car's centre of mass rather than the rear axle (`ComputeCOMPosition`), the choice of Section 5.2.

Both stacks then hand these errors to their tracker: Autoware to model predictive control, which re-plans the next few seconds of steering at every step, or to pure pursuit, and Apollo to a linear-quadratic regulator, a feedback law whose gains come from an optimisation. The next Notes build the simpler trackers first.

7. Summary

Quantity	What it says	At the car (14.0, 0.0) heading 10°
Nearest path point P	the course point closest to the rear axle	(13.714, 0.715)
Arc length s	how far along the course P is	13.81 m
Lateral error e	how far to the side of the course, + left	−0.770 m
Heading error θ_e	heading minus the course's direction at P	−11.80°
Along-track error	how far behind a timetable (trajectories only)	3.0 m after a 1.5 s hold

- The opening question has its answer: the car finds, many times a second, the nearest point of the course and measures there its distance along the course, its lateral error and its heading error; steering on the last two, the car of Figure 1 follows the whole course in 18.15 s, where steering on one fixed line left it 5.57 m off after 11.0 s, because the line it steers on now moves with it.
- The **kinematic bicycle model** (G-3128) moves the car's rear axle along its heading and turns the heading at v times the tangent of the **steering angle** (G-3126) divided by the **wheelbase** (G-3127), because the car turns about a point on its rear axle's line; it is enough here because our sideways accelerations stay at or below 2.5 m/s^2, about half the 4.91 m/s^2 limit.
- **Path curvature** (G-3129) κ, 1 divided by the radius, says how sharply a path bends, so it fixes the steering a bend needs: the tangent of δ is L times κ, 14.04° on the 10 m arc.
- **Path following** (G-3130) keeps the car near a path at its own speed, so a 1.5 s delay just makes it 1.80 s late; **trajectory tracking** (G-3131) also chases a timetable, so it measures an **along-track error** (G-3137) and speeds up to catch it.
- **Path coordinates** (G-3132), measured from the **nearest path point** (G-3134) in its Frenet frame, give the **arc length** (G-3133) s, the **lateral error** (G-3135) and the **heading error** (G-3136), so a tracker on any path can steer as if on a straight line.
- A tracker needs both errors, because the lateral error acts as the spring and the heading error as the damper: with only one, the car weaves up to 2.09 m off or stays 1.00 m off.
- The errors depend on the point of the car they are measured at, because the heading error moves the front axle relative to the rear one: 0.737 m at the rear axle is 0.271 m at the front.
- A tracker that steers on errors alone keeps a steady error on a bend, here near −0.39 m, because the turn the arc needs has to come from an error.

The car can now say, at every moment, how far it is from its path and which way it points compared with it. Still missing: a steering law that uses these errors well, one that does not keep 0.39 m outside a bend and copes with higher speed. The first, pure pursuit, steers towards a point on the path ahead of the car.

8. Sources

Built from

- NPTEL-NOC IITM, *Wheeled Mobile Robots*, lecture 34, "Introduction to Motion Control of Mobile Robots Part 2", YouTube, https://www.youtube.com/watch?v=jXTtnbOHT7I (NPTEL L34)
- Snider, J. M. (2009). *Automatic Steering Methods for Autonomous Automobile Path Tracking*, CMU-RI-TR-09-08, Carnegie Mellon University. §2.1 "Geometric Vehicle Model", §2.2 "Pure Pursuit", §2.3 "Stanley Method", §3.1 "Kinematic Bicycle Model", §3.1.1 "Path Coordinates", https://www.ri.cmu.edu/pub_files/2009/2/Automatic_Steering_Methods_ (Snider)
- Gupta, A. and Srinivasa, S., CSE 478 *Autonomous Robotics*, Lecture 10 "Intro to Control", University of Washington, 2026, https://courses.cs.washington.edu/courses/cse478/26sp/lectures (UW CSE 478 L10)
- Paden, B., Čáp, M., Yong, S. Z., Yershov, D. and Frazzoli, E. (2016). "A Survey of Motion Planning and Control Techniques for Self-driving Urban Vehicles", *IEEE Transactions on Intelligent Vehicles* 1(1). §III vehicle models, §V vehicle control (Problems V.1 and V.2, eq. V.10), https://arxiv.org/abs/1604.07446 (Paden)
- Lynch, K. M. and Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press. §13.3.1 the car's turning radius from its steering angle. Free preprint: http://modernrobotics.org (MR)

Other references

- Polack, P., Altché, F., d'Andréa-Novel, B. and de La Fortelle, A. (2018). "Guaranteeing Consistency in a Motion Planning and Control Architecture Using a Kinematic Bicycle Model", https://arxiv.org/abs/1804.08290 (the 0.5 μg limit of the kinematic bicycle model)
- OpenStax, *University Physics Volume 1*, §4.4 "Uniform and Nonuniform Circular Motion" (centripetal acceleration, eq. 4.27) and §6.2 "Friction", Table 6.1 (rubber on dry concrete), https://openstax.org/books/university-physics-volume-1 (OpenStax)
- MathWorks, Navigation Toolbox, `referencePathFrenet` documentation (Frenet states: arc length and perpendicular deviation from the reference path), https://www.mathworks.com/help/nav/ref/referencepathfrenet.html
- Autoware Foundation, `autoware_universe`, package `autoware_mpc_lateral_controller` under `control`: `src/mpc_utils.cpp` (`calcNearestPoseInterp`, `calcLateralError`), `src/mpc.cpp`, `src/mpc_lateral_controller.cpp` (`trajectory_reference_mode`); `autoware_core`, `common/autoware_motion_utils/README.md` (nearest search with distance and yaw thresholds), https://github.com/autowarefoundation (read for Section 6)
- Apollo Auto, `apollo`, under `modules/control`: the file `lat_controller.cc` (`ComputeLateralErrors`) in `controllers/lat_based_lqr_controller`, and the file `trajectory_analyzer.cc` (`QueryNearestPointByPosition`) in `control_component/controller_t` https://github.com/ApolloAuto/apollo (read for Section 6)

9. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Steering angle δ (G-3126)	The angle between a car's steered front wheel and its heading, positive to the left, such as at most 26.6° for RO-055's car; it sets how sharply the car turns, through $\tan\delta = L/R$.
Wheelbase L (G-3127)	The distance from a car's rear axle to its front axle, such as 2.5 m; with the steering angle it fixes the turning radius, $R = L/\tan\delta$, so a longer car turns more widely for the same steering.
Kinematic bicycle model (G-3128)	A car drawn as one steered front wheel and one rear wheel on its centre line, with no tyre slip: the rear axle moves along the heading and the heading turns at $\dot{\theta} = v\tan\delta/L$; it predicts a car's motion well while the sideways acceleration stays below about half the road's friction (grip) coefficient times gravity.
Path curvature κ (G-3129)	How fast a path's direction turns per metre along it, $1/R$ on an arc of radius R, such as 0.1 per m on a 10 m arc and 0 on a straight; it fixes the steering a bend needs, $\tan\delta = L\kappa$.
Path following (G-3130)	Keeping a vehicle close to a path, a curve with no clock, at a speed of its own choosing, like a car on a road; a delay only makes it arrive later, so it needs only the lateral and heading errors.
Trajectory tracking (G-3131)	Making a vehicle be at each point of a timed trajectory at its time, like a train meeting its signals; a delay leaves an along-track error, which the vehicle must make up by changing speed.
Along-track error (G-3137)	How far behind (or ahead of) its timed reference a vehicle is, measured along the path, such as 3.0 m after a 1.5 s hold against a 2 m/s timetable; trajectory tracking drives it to zero by changing speed, path following ignores it.

Term	Meaning
Nearest path point (G-3134)	The point of a path closest to a chosen point of the vehicle, such as its rear axle; trackers measure the lateral and heading errors there, and on a path that comes back near itself the plain nearest point can jump, so real stacks limit the search.
Arc length s (distance along a path) (G-3133)	The distance walked along a path from its start to a given point, such as 13.81 m to a point 21.8° round the course's arc; it labels every point of the path, like the kilometre stones of a road.
Path coordinates (Frenet frame) (G-3132)	Giving a point's place by the distance along a path to its nearest path point and the signed distance to the side of it, measured along the path's tangent and normal there, the Frenet frame that slides along the path; they let a tracker treat any bending path like a straight line.
Lateral error e (cross-track error) (G-3135)	The signed distance from a path to a point of the vehicle, measured along the path's normal at the nearest path point and positive to the left, such as -0.770 m for a car outside an arc; it is one of the two numbers a path tracker drives to zero.
Heading error θ_e (G-3136)	A vehicle's heading minus the path's direction at the nearest path point, wrapped into $-180°$ to $180°$, such as $10° - 21.8° = -11.8°$; it says whether the vehicle is turning back to the path or away, and the lateral error changes at $v \sin \theta_e$.
Instantaneous centre of curvature (ICC) (G-2529)	The point a turning robot's whole body circles at a given moment; for a differential drive it lies on the axle line at distance $R = v/\omega$ from the midpoint, so it fixes how fast each wheel must move.
Turning radius R (G-2293)	The distance from the point a robot is turning about (the instantaneous centre of curvature, ICC) to the robot's reference point; $R = v/\omega$, so it says how tight a turn is.

Term	Meaning
Linear velocity v (G-2291)	How fast a robot's reference point (for a differential drive, the middle of the axle) moves along its heading, in m/s; for a differential drive, the average of the two wheel speeds, so it sets how fast the robot advances.
Dot notation $\dot{x}$ (G-2296)	A dot over a quantity means its rate of change per second (its derivative with respect to time), such as $\dot{x} = 0.433$ m/s; models of motion are written with it.
Kinematic model (G-2297)	Equations that give how a robot's configuration changes from its speed inputs alone, ignoring masses and forces, such as $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$, $\dot{\theta} = \omega$; used to predict motion at low speed.
Trajectory (G-2886)	The state of a robot at every moment of a time interval, together with the controls that produce it, such as a local planner's band of poses with times; unlike a path, it says when the robot is where and how fast it moves.
Tracking (G-2954)	Making an output follow a reference that changes over time, such as a planned speed rising at 0.25 m/s^2; also called the servo problem, it needs the controller to keep up with the change, and a correction from the error alone lags behind.
Regulation (G-2952)	Keeping an output close to a fixed reference in spite of disturbances, such as holding 0.5 m/s on a ramp; the controller's job is to undo whatever pushes the output away.
Angle wrapping (G-2404)	Adding or subtracting full turns until an angle or angle difference lies between $-180°$ and $180°$, so that $175°$ and $-178°$ count as $7°$ apart, not $353°$; needed whenever bearings or headings are compared.
Steady-state error (G-2985)	The command minus the value a step response settles at, such as 0.125 m/s when a speed loop asked for 0.5 m/s settles at 0.375 m/s; a P loop keeps one whenever it must keep pushing against a load.